

TECHNISCHE DOKUMENTATION

Entwickler-Dokumentation

Analyse, SQL-Ermittlung, Entwicklung und Serverbereitstellung

Projekt: LOGINventoryHardware

Zielgruppe: Entwicklung und technische Weiterentwicklung

Umgebung: LOGINventory9, IIS, ASP.NET Core 8, SQL Server

Stand: 05.09.2026

Verantwortlich: Petru Gavriluc

1. Ausgangslage und Analyse

LOGINventory zeigte allgemeine Assetdaten, aber die gewünschten angeschlossenen Komponenten nicht direkt in der Detailansicht. In der LOGINventory-Datenbank waren die Informationen vorhanden. Ziel war eine Erweiterung ohne Änderung der kompilierten LOGINventory-DLL.

2. Ermittlung der Datenbankstruktur

Die Tabellen und Spalten wurden zunächst lesend in SQL Server Management Studio untersucht. Die Inventarnummer dient als fachlicher Einstieg, während die interne Beziehung über die Asset-ID erfolgt.

Tabelle	Verwendung
dbo.Asset	Grunddaten und interne ID des Assets
dbo.HardwareAsset	Inventarnummer, Seriennummer und benutzerdefinierter Typ
dbo.Device	DNS-Hostname und gerätespezifische Informationen
dbo.MonitorInfo / dbo.Monitor	Verknüpfung und Stammdaten der Monitore
dbo.DockingStationInfo / dbo.DockingStation	Strukturierte Dockingstationinformationen, soweit vorhanden
dbo.UsbDeviceInfo / dbo.UsbDevice	USB-Geräte, Drucker, Kameras und technische Unterkomponenten

Beispiel: Asset ermitteln

```
SELECT TOP (1)
    a.Id AS AssetId,
    a.Name,
    ha.InventoryNumber,
    ha.SerialNumber,
    ha.CustomType,
    d.DnsHostName
FROM dbo.Asset AS a
INNER JOIN dbo.HardwareAsset AS ha ON ha.Id = a.Id
INNER JOIN dbo.Device AS d ON d.Id = a.Id
WHERE ha.InventoryNumber = @InventoryNumber
      AND a.Archived IS NULL;
```

Beispiel: Monitore laden

```
SELECT
    mi.Ordinal, m.Name, m.Model, m.Vendor, m.Size,
    mi.SerialNumber, mi.ManufactureWeek,
    mi.ConnectionType, mi.Timestamp AS LastInventory
FROM dbo.MonitorInfo AS mi
INNER JOIN dbo.Monitor AS m ON m.Id = mi.ItemId
WHERE mi.AssetId = @AssetId
    AND mi.Archived = 0
ORDER BY mi.Ordinal;
```

Wichtig: Die tatsächlich vorhandenen Spalten wurden gegen das Schema geprüft. Die Verknüpfung bei HardwareAsset und Device erfolgte über `Id`, nicht über eine angenommene Spalte `AssetId`.

3. Entwicklung der ASP.NET-Core-API

Es wurde ein schlankes ASP.NET-Core-8-Projekt angelegt. Die API besteht aus Models, einer SQL-Verbindungsfactory, einem Repository und einem API-Controller.

```
LOGINventoryHardware
├─ Controllers/HardwareApiController.cs
├─ Data/SqlConnectionFactory.cs
├─ Data/LoginventoryRepository.cs
├─ Models/AssetInfo.cs
├─ Models/MonitorInfo.cs
├─ Models/DockingStationInfo.cs
├─ Models/UsbDeviceInfo.cs
├─ Models/ConnectedHardwareResult.cs
├─ Program.cs
└─ appsettings.json
```

Registrierung in Program.cs

```
builder.Services.AddControllers();
builder.Services.AddSingleton<SqlConnectionFactory>();
builder.Services.AddScoped<LoginventoryRepository>();
app.MapControllers();
```

API-Vertrag

```
GET /api/hardware?invnr=ASSET-3
```

Die Antwort enthält die Objekte `asset`, `monitors`, `dockingStations` und `usbDevices`. Nullable Zeitstempel werden durchgängig als `DateTime?` modelliert.

4. Lokaler Test

1. Projekt kompilieren: `dotnet build`
2. Projekt starten: `dotnet run`

3. API aufrufen: `https://localhost:7024/api/hardware?invnr=ASSET-3`
4. JSON auf Asset, Monitore und USB-Geräte prüfen

5. Veröffentlichung auf IIS

1. Release-Veröffentlichung für `net8.0` erstellen.
2. Dateien nach `C:\inetpub\wwwroot\LOGINventoryHardware` kopieren.
3. Eigenen Anwendungspool `LOGINventoryHardware` mit „Kein verwalteter Code“ anlegen.
4. Ordner als IIS-Anwendung mit Alias `LOGINventoryHardware` konfigurieren.
5. .NET 8 Hosting Bundle und `AspNetCoreModuleV2` prüfen.
6. Produktive API testen.

```
dotnet --list-runtimes
Test-Path "C:\Program Files\IIS\Asp.Net Core Module\V2\aspnetcorev2.dll"
```

6. SQL-Berechtigungen

Bei `Integrated Security=True` greift der IIS-Anwendungspool unter seiner Identität auf SQL Server zu. Der Login und Datenbankbenutzer wurden eingerichtet und mit Leserechten versehen.

```
CREATE LOGIN [IIS APPPOOL\LOGINventoryHardware] FROM WINDOWS;
USE [LOGINventory9];
CREATE USER [IIS APPPOOL\LOGINventoryHardware]
FOR LOGIN [IIS APPPOOL\LOGINventoryHardware];
ALTER ROLE [db_datareader]
ADD MEMBER [IIS APPPOOL\LOGINventoryHardware];
```

7. Integration in LOGINventory9

Die Datei `DeviceDetailViewControl.ascx` wurde gesichert und um drei Elemente erweitert:

1. HTML-Button ohne `runat="server"`
2. verdeckter Inhaltsbereich für die Hardwaredarstellung
3. JavaScript für URL-Auswertung, API-Aufruf, Filterung und DOM-Erzeugung

Der Button ist absichtlich kein ASP.NET-Servercontrol. Dadurch ist keine Anpassung der kompilierten LOGINventory-Assembly erforderlich.

8. Filter- und Darstellungslogik

- Monitore werden separat dargestellt.
- Unter „Geräte“ werden nur Dockingstationen und Drucker hervorgehoben.
- Audio-Unterkomponenten wie USB Audio, Headset oder Mikrofon werden nicht als Dockingstation gewertet.
- Andere technische Komponenten verbleiben unter „USB-Geräte anzeigen“.

- Umlaute werden in JavaScript über Unicode-Escapes wie `ä` und `ö` kodierungssicher ausgegeben.
- Fehlende Werte werden als Unicode-Gedankenstrich `–` dargestellt.

9. Fehleranalyse

Fehlerbild	Ursache / Lösung
HTTP 500.19 / 0x8007000d	ASP.NET Core Module fehlte. .NET 8 Hosting Bundle installieren und IIS neu starten.
404 unter IIS	Ordner war nicht als IIS-Anwendung konfiguriert oder falscher Anwendungspfad.
API lokal, aber nicht auf Server	Anwendungspool, Hosting Bundle, web.config und SQL-Rechte prüfen.
Klick ohne Reaktion	Unvollständiger JavaScript-Block oder Funktion nicht global verfügbar. Browserkonsole prüfen.
Geräte / Zubehör / €	Zeichenkodierung. Unicode-Escapes in dynamischen JavaScript-Texten verwenden.

10. Weiterentwicklung

- Gezielte SQL-Views statt direktem Tabellenzugriff
- Authentifizierung und Autorisierung der API
- Serverseitige Klassifikation statt JavaScript-Filterung
- Versionierung der ASCX-Anpassung
- Automatisierte Integrationstests für API und SQL-Mapping